PRO2000 - PROGRAMMING

Week 37- Functions

Gebremariam Assres
Gebremariam.assres@kristiania.no

Today's topics

- Modular programming and functions
- Function prototypes and parameters
- Passing data by value and reference
- Return values and function exit ()
- Local, global, and static variables
- Default arguments

Modular Programming

- Modular programming:
  - breaking a program up into smaller, manageable functions or modules.
  - supports the divide-and-conquer approach to solving a problem.
- Function:
  - a collection of statements to perform a specific task
- Motivation for modular programming
  - simplifies the process of writing programs
  - improves maintainability of programs

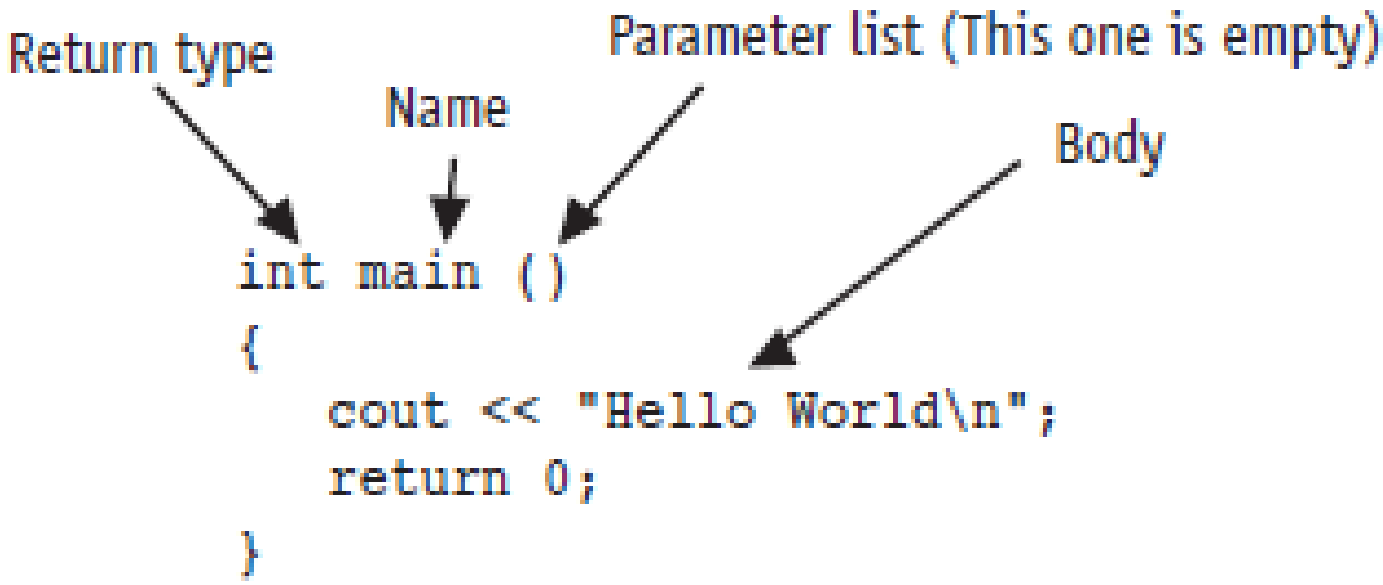
Defining and Calling Functions

- Function call:
  - a statement that causes a function to execute
- Function definition:
  - the statements that make up a function

Function Definition

- Definition includes
  - name: name of the function. Function names follow same rules as variable names
  - parameter list: variables that hold the values passed to the function
  - body: statements that perform the function's task
  - return type: data type of the value the function returns to the part of the program that called it

Function Definition ...



Function Header

- The function header consists of
  - the function *return type*
  - the function *name*
  - the function *parameter list*
- Example:

```
int main()
```

- Note: no `;` at the end of the header

Function Return Type

- If a function returns a value, the type of the value must be indicated

```
int main()
```

- If a function does not return a value, its return type is **void**

```
void printHeading()
{
   cout << "\tMonthly Sales\n";
}
```

Calling a Function

- To call a function, use the function name followed by `()` and `;`

```
printHeading();
```

- When a function is called, the program executes the body of the function
- After the function terminates, execution resumes in the calling module at the point of call

Calling a Function

- `main` is automatically called when the program starts
- `main` can call any number of functions
- Functions can call other functions

Function Prototypes

- The compiler must know the following about a function before it is called
  - name
  - return type
  - number of parameters
  - data type of each parameter

Function Prototypes

Ways to notify the compiler about a function before a call to the function:

- Place function definition before calling function's definition
- Use a function prototype (similar to the heading of the function
  - Heading: `void printHeading()`
  - Prototype: `void printHeading();`
- Function prototype is also called a function declaration

Sending Data into a Function

- Can pass values into a function at time of call

```
c = sqrt(a*a + b*b);
```

- **Values** passed to function are arguments
- **Variables** in function that hold values passed as arguments are parameters
- Alternate names:
  - argument: actual argument, actual parameter
  - parameter: formal argument, formal parameter

Parameters, Prototypes, Function Headings

- For each function argument,
  - the prototype must include the data type of each parameter in its `()`

```
void evenOrOdd(int); //prototype
```

  - the heading must include a declaration, with variable type and name, for each parameter in its `()`

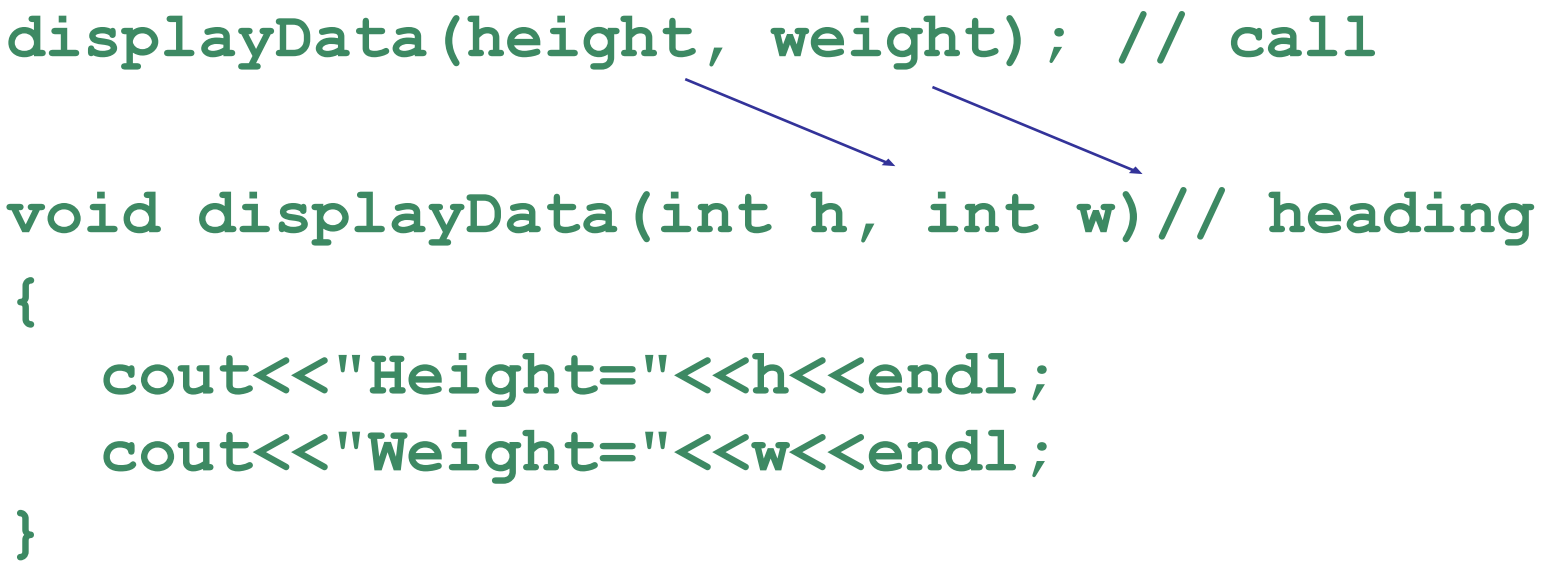
```
void evenOrOdd(int num) //heading
```

- The function call for the above function would look like
  - **this:** `evenOrOdd(val); //call`
  - **Note: no data type on argument in call**

Calling Functions with Multiple Arguments

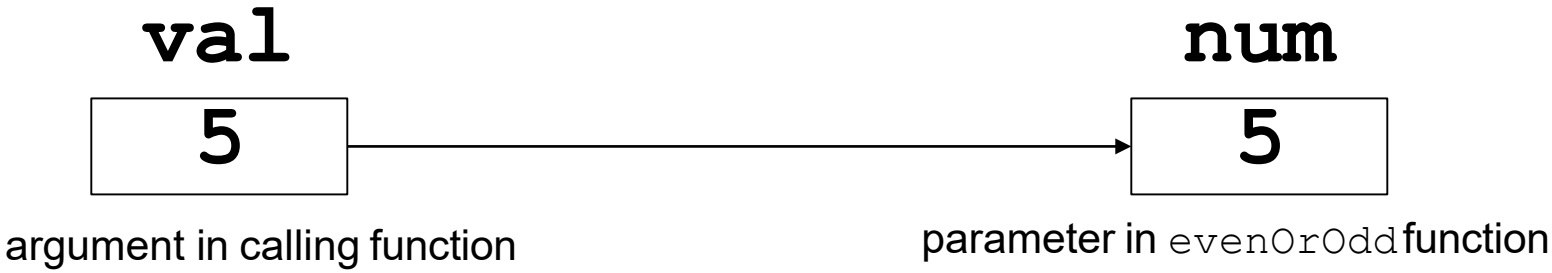
When calling a function with multiple arguments

- the number of arguments in the call must match the function prototype and definition
- the first argument will be copied into the first parameter, the second argument into the second parameter, etc.



Passing Data by Value

- Pass by value:
  - When an argument is passed to a function, a copy of its value is placed in the parameter
  - The function cannot access the original argument
- Changes to the parameter in the function do not affect the value of the argument in the calling function
- Example: `int val = 5; evenOrOdd(val);`
- **`evenOrOdd`** can change variable **`num`**, but it will have no effect on variable **`val`**



The `return` Statement

- Used to end execution of a function to return a value to the caller
- Can be placed anywhere in a function
  - Statements that follow the **return** statement will not be executed
  - Can be used to prevent abnormal termination of program
- Without a `return` statement, the function ends at its last `}`
- Prototype and definition must indicate data type of return value (not **void**) and calling function should use return value, *e.g.*,
  - assign it to a variable, send it to **cout**, use it in an arithmetic computation, use it in a relational expression

The **`return`** Statement ...

- Format: **return *expression;***
- ***expression*** may be a variable, a literal value, or an expression.
- ***expression*** should be of the same data type as the declared return type of the function (will be converted if not)
- Function can return true or false, declare the return type in the function prototype and heading as **bool**
- The function body must contain **return** statement(s) that return true or false
- The calling function can use the return value in a relational expression

The `return` Statement ...

Boolean `return` Example

```
bool isValid(int);                  // prototype

bool isValid(int val)               // heading
{
    int min = 0, max = 100;
    if (val >= min && val <= max)
        return true;
    else
        return false;
}

if (isValid(score))                 // call
    ...
```

Local Variables

- Local variable is defined within a function or block, accessible only within the function or block, other functions and blocks can define variables with the same name.

- When a function is called, local variables in the calling function are not accessible from within the called function.

- A local variable only exists while its defining function is executing
- Local variables are destroyed when the function terminates

- Data cannot be retained in local variables between calls to the function in which they are defined

Global Variables

- Global variable is a variable defined outside all functions; it is accessible to all functions within its scope

- Easy way to share large amounts of data between functions

- Scope of a global variable is from its point of definition to program end
- Use sparingly

Initializing Local and Global Variables

- Local variables must be initialized by the programmer
- Global variables are initialized to `0` (numeric) or **`NULL`** (character) when the variable is defined.
  - These can be overridden with explicit initial values.

Global Variables – Why Use Sparingly?

Global variables make:

- Programs that are difficult to debug
- Functions that cannot easily be re-used in other programs
- Programs that are hard to understand

Global Constants

- A global constant is a named constant that can be used by every function in a program
- It is useful if there are unchanging values that are used throughout the program
- They are safer to use than global variables, since the value of a constant cannot be modified during program execution

Local and Global Variable Names

- Local variables can have same names as global variables
- When a function contains a local variable that has the same name as a global variable, the global variable is unavailable from within the function.
- The local definition "hides" or "shadows" the global definition.

Static Local Variables

- Local variables
  - Only exist while the function is executing
  - Are redefined each time function is called
  - Lose their contents when function terminates
- **`static`** local variables
  - Are defined with key word **`static`** `static int counter;`
  - Are defined and initialized only the first time the function is executed
  - Retain their contents between function calls

Default Arguments

- Values passed automatically if arguments are missing from the function call
- Must be a constant declared in prototype or header (whichever occurs first)

```
void evenOrOdd(int = 0);
```

- Multi-parameter functions may have default arguments for some or all parameters

```
int getSum(int, int=0, int=0);
```

Default Arguments ...

- If not all parameters to a function have default values, the ones without defaults must be declared first in the parameter list

```
int getSum(int, int=0, int=0);// OK
int getSum(int, int=0, int);  // wrong!
```

- When an argument is omitted from a function call, all arguments after it must also be omitted

```
sum = getSum(num1, num2);     // OK
sum = getSum(num1, , num3);   // wrong!
```

Using Reference Variables as Parameters

- Mechanism that allows a function to work with the original argument from the function call, not a copy of the argument
- Allows the function to modify values stored in the calling environment
- Provides a way for the function to ‘return’ more than 1 value

Reference Variables

- A reference variable is an alias for another variable
  - It is defined with an ampersand (`&`) in the prototype and in the header

```
void getDimensions(int&, int&);
```

- Changes to a reference variable are made to the variable it refers to
- Use reference variables to implement passing parameters by reference

Pass by Reference Example

```
void squareIt(int &); //prototype
void squareIt(int &num)
{
     num *= num;
}

int localVar = 5;
squareIt(localVar);   // localVar now
                      // contains 25
```

Reference Variable Notes

- Each reference parameter must contain **&**
- Argument passed to reference parameter must be a variable. It cannot be an expression or a constant.
- Use only when appropriate, such as when the function must input or change the value of the argument passed to it
- Files (*i.e.*, file stream objects) should be passed by reference

The `exit()` Function

- Terminates execution of a program
- Can be called from any function
- Can pass a value to OS to indicate status of program execution
- Usually used for abnormal termination of program
- Requires `cstdlib` header file

The `exit()` Function ...

- Use an integer value to indicate program status
- Often, 0 means successful completion, non-zero indicates a failure condition
- Can use named constants defined in cstdlib: EXIT_SUCCESS and EXIT_FAILURE

Up next: Exercises

- Now: 2 hours (12:15 to 14:00) of exercises (found on Canvas).

- Short exercise summary:
  - *Write, compile, and debug a C++ programs with emphasis on functions.*
- Remember:
  - You should do approximately 4 more hours of the course (in a study group or alone) *before* our next session together. :-)